

CFRM 421/521

[Eunice Seung-hyun Han]

Homework 1

- **Due: Tuesday, April 15, 2025, 11:59 PM**
- Total marks: 49
- Late submissions are allowed, but a 20% penalty per day applies. Your last submission is considered for calculating the penalty.
- Use this Jupyter notebook as a template for your solutions. **Your solution must be submitted as both one Jupyter notebook and one PDF file on Gradescope.** There will be two modules on Gradescope, one for each file type. The notebook must be already run, that is, make sure that you have run all the code, save the notebook, and then when you reopen the notebook, checked that all output appears as expected. You are allowed to use code from the textbook, textbook website, or lecture notes.

1. Preprocessing housing data [10 marks]

Hint: Reading Chapter 2 of the textbook and the accompanying [Jupyter notebook](#) on the textbook website may help with this question.

Consider the California housing data from Chapter 2 of the textbook. The following code fetches the data.

```
In [ ]: from pathlib import Path
import pandas as pd
import tarfile
import urllib.request
import numpy as np
from sklearn.model_selection import StratifiedShuffleSplit
import sys

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
```

```
if not tarball_path.is_file():
    Path("datasets").mkdir(parents=True, exist_ok=True)
    url = "https://github.com/ageron/data/raw/main/housing.tgz"
    urllib.request.urlretrieve(url, tarball_path)
    with tarfile.open(tarball_path) as housing_tarball:
        housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))
```

```
housing = load_housing_data()
```

```
In [ ]: print(housing)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedr
0	-122.23	37.88	41.0	880.0	1
1	-122.22	37.86	21.0	7099.0	11
2	-122.24	37.85	52.0	1467.0	1
3	-122.25	37.85	52.0	1274.0	2
4	-122.25	37.85	52.0	1627.0	2
...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	3
20636	-121.21	39.49	18.0	697.0	1
20637	-121.22	39.43	17.0	2254.0	4
20638	-121.32	39.43	18.0	1860.0	4
20639	-121.24	39.37	16.0	2785.0	6

	population	households	median_income	median_house_value
0	322.0	126.0	8.3252	452600.0
1	2401.0	1138.0	8.3014	358500.0
2	496.0	177.0	7.2574	352100.0
3	558.0	219.0	5.6431	341300.0
4	565.0	259.0	3.8462	342200.0
...	...	...	...	...
20635	845.0	330.0	1.5603	78100.0
20636	356.0	114.0	2.5568	77100.0
20637	1007.0	433.0	1.7000	92300.0
20638	741.0	349.0	1.8672	84700.0
20639	1387.0	530.0	2.3886	89400.0

	ocean_proximity
0	NEAR BAY
1	NEAR BAY
2	NEAR BAY
3	NEAR BAY
4	NEAR BAY
...	...
20635	INLAND
20636	INLAND
20637	INLAND
20638	INLAND
20639	INLAND

[20640 rows x 10 columns]

Next, we create stratified test and training sets based on different income categories.

```
In [ ]: housing["income_cat"] = pd.cut(housing["median_income"],
                                       bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                       labels=[1, 2, 3, 4, 5])

split_obj = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_s
for train_index, test_index in split_obj.split(housing, housing["incom
    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]

for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

Let us separate the features (`X_raw`) and the response variable (`y`).

```
In [ ]: X_raw = strat_train_set.drop("median_house_value", axis=1)
y = strat_train_set["median_house_value"].copy()
```

(a) Handling missing values [1 mark]

Let us find out if there are any missing values in our features. As the code below indicates, out of 16512 observations, 158 have missing values. Further inspection shows that `total_bedrooms` has missing values (only a few missing values are shown below).

```
In [ ]: X_raw.shape
```

```
Out[ ]: (16512, 9)
```

```
In [ ]: y.shape
```

```
Out[ ]: (16512,)
```

```
In [ ]: X_raw[X_raw.isnull().any(axis=1)].shape
```

```
Out[ ]: (168, 9)
```

```
In [ ]: X_raw[X_raw.isnull().any(axis=1)].head()
```

Out []:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
14452	-120.67	40.50	15.0	5343.0	NaN
18217	-117.96	34.03	35.0	2093.0	NaN
11889	-118.05	34.04	33.0	1348.0	NaN
20325	-118.88	34.17	15.0	4260.0	NaN
14360	-117.87	33.62	8.0	1266.0	NaN

Task: Read the subsection "Data Cleaning" in Chapter 2 of the textbook. Use the `sklearn.impute.SimpleImputer` class to fill the missing values in the numerical features (everything except `ocean_proximity`) with the median of the corresponding feature.

Solution:

```
In [ ]: from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
X_raw_num = X_raw.select_dtypes(include=[np.number])
imputer.fit(X_raw_num)
imputer.statistics_
X_raw_num.median().values
X_imputer = imputer.transform(X_raw_num)
X_tr = pd.DataFrame(X_imputer, columns=X_raw_num.columns,
                    index=X_raw_num.index)

X_tr.loc[X_raw.isnull().any(axis=1)].head()
print("Result of filling the missing values in the numerical features")
print(X_tr)

# This shows the data cleaning process had worked well as there is no
#X_clean = pd.concat([X_tr, X_raw["ocean_proximity"]],axis=1)
#X_clean.loc[X_raw.isnull().any(axis=1)].head()
```

Result of filling the missing values in the numerical features with the median of the corresponding feature:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
13096	-122.42	37.80	52.0	3321.0	11
14973	-118.38	34.14	40.0	1965.0	3
3785	-121.98	38.36	33.0	1083.0	2
14689	-117.11	33.75	17.0	4174.0	8
20507	-118.15	33.77	36.0	4366.0	12
...	...	...	...	...	...
14207	-118.40	33.86	41.0	2237.0	5
13105	-119.31	36.32	23.0	2945.0	5
19301	-117.06	32.59	13.0	3920.0	7
19121	-118.40	34.06	37.0	3781.0	8
19888	-122.41	37.66	44.0	431.0	1

	population	households	median_income
13096	1576.0	1034.0	2.0987
14973	666.0	357.0	6.0876
3785	562.0	203.0	2.4330
14689	1845.0	780.0	2.2618
20507	1912.0	1172.0	3.5292
...	...	...	...
14207	938.0	523.0	4.7105
13105	1419.0	532.0	2.5733
19301	2814.0	760.0	4.0616
19121	1725.0	838.0	4.1455
19888	682.0	212.0	3.2833

[16512 rows x 8 columns]

(b) Handling categorical features [2 marks]

ML algorithms generally work with numerical values. So, we need to convert categorical features to numerical values. One way is to simply map each category to an integer. Another alternative is to use **one-hot encoding**.

Task: Read subsection "Handling Text and Categorical Attributes" in Chapter 2 of the textbook. Then use the `sklearn.preprocessing.OrdinalEncoder` class

and the `sklearn.preprocessing.OneHotEncoder` class to transform the `ocean_proximity` feature to integers and one-hot vectors. Explain why it is more appropriate to use one-hot encoding.

Solution:

```
In [ ]: X_raw_ocean = X_raw[["ocean_proximity"]] # Or maybe use X_clean
```

```
# Using OrdinalEncoder
```

```
from sklearn.preprocessing import OrdinalEncoder
```

```
ordinal_encoder = OrdinalEncoder()
```

```
X_raw_ocean_encoded = ordinal_encoder.fit_transform(X_raw_ocean)
```

```
print("Result of using OrdinalEncoder class: ")
```

```
print(X_raw_ocean_encoded)
```

```
print("Categories: ", ordinal_encoder.categories_)
```

Result of using OrdinalEncoder class:

```
[[3.]
```

```
 [0.]
```

```
 [1.]
```

```
 ...
```

```
 [4.]
```

```
 [0.]
```

```
 [4.]]
```

```
Categories: [array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
                  dtype=object)]
```

```
In [ ]: # Using OneHotEncoder
```

```
from sklearn.preprocessing import OneHotEncoder
```

```
onehot_encoder = OneHotEncoder(sparse_output=False) # use sparse_output
```

```
X_raw_ocean_1hot = onehot_encoder.fit_transform(X_raw_ocean)
```

```
print("Result of using OneHotEncoder class: ")
```

```
print(X_raw_ocean_1hot) # sparse matrix that save plenty of memory and
```

```
print("Categories: ", onehot_encoder.categories_)
```

Result of using OneHotEncoder class:

```
[[0. 0. 0. 1. 0.]
```

```
 [1. 0. 0. 0. 0.]
```

```
 [0. 1. 0. 0. 0.]
```

```
 ...
```

```
 [0. 0. 0. 0. 1.]
```

```
 [1. 0. 0. 0. 0.]
```

```
 [0. 0. 0. 0. 1.]]
```

```
Categories: [array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
                  dtype=object)]
```

The reason using one-hot encoding is more appropriate is because

OrdinalEncoder representation would cause issue as Machine Learning algorithms will assume the nearby values are more similar than two distant values. For example, it will assume categories 0 and 1 are more similar than categories 0 and 4. This is not the right approach in "ocean_proximity" column in this data set. One-hot encoding effectively solves this issue as it creates one binary attribute to each category which means it allows only one attribute to equal 1 and otherwise, it would be 0. For example, the attribute would equal to 1 when the category is "INLAND" and otherwise it will be 0. One-hot encoding does this for each category and in Scikit-Learn, the OneHotEncoder class would convert the categories into one-hot vectors. One-hot encoding successfully converts categorical features into numerical values better than Ordinal encoding as it does not cause confusion to ML algorithms.

(c) Feature engineering [2 marks]

Often it may be useful to experiment with transforming and combining features to create new features that may be expected to better predict the target variable. Features that are skewed or heavily tailed can be logged to make them closer to normal. An example of a feature combination is instead of using the number of bedrooms, which may not be meaningful by itself, we should consider the number of bedrooms relative to the number of rooms.

Task: Read the subsections "Custom Transformers" and "Transformation Pipelines" in Chapter 2 of the textbook. Use the `sklearn.preprocessing.FunctionTransformer` class take the log of `population`, and to create the ratio `total_bedrooms/total_rooms`. Print these two transformed features.

Solution:

```
In [ ]: from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(X_raw[["population"]])
print("Log of Population:")
print(log_pop)

ratio_bedrooms_rooms = FunctionTransformer(lambda X: np.asarray(X)[:,
ratio_bedrooms_rooms_result = ratio_bedrooms_rooms.transform(X_raw[["t
print("Ratio of total_bedrooms/total_rooms:")
print(ratio_bedrooms_rooms_result)
```

Log of Population:

	population
13096	7.362645
14973	6.501290
3785	6.331502
14689	7.520235
20507	7.555905
...	...
14207	6.843750
13105	7.257708
19301	7.942362
19121	7.452982
19888	6.525030

[16512 rows x 1 columns]

Ratio of total_bedrooms/total_rooms:

```
[[0.33574225]
 [0.18015267]
 [0.20036934]
 ...
 [0.19770408]
 [0.2308913 ]
 [0.45243619]]
```

(d) Feature scaling and transformation [1 mark]

Many ML algorithms do not perform well if the features have different scales (some very small, and others very large). In such scenarios, one should transform the features so that they have a similar range of values. There are two common ways to achieve this. In **min-max** scaling, we subtract the feature from its minimum value and then divide by its range (i.e. maximum value minus minimum value) so that the scaled values are between 0 and 1. In **standardization**, we subtract the values from the average and divide by the standard deviation, so that the transformed values has mean 0 and variance 1, here we focus on the latter.

Task: Read the subsection "Feature Scaling and Transformation" in Chapter 2 of the textbook. Use the `sklearn.preprocessing.StandardScaler` class to scale the numerical features using standardization.

Solution:

```
In [ ]: from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
X_raw_num_std_scaled = std_scaler.fit_transform(X_raw_num)
# print(X_raw_num_std_scaled)

X_raw_num_std_scaled = pd.DataFrame(X_raw_num_std_scaled, columns=X_ra
```

```

index=X_raw_num.index)

print("Result of scaling the numerical features using standardization:
print(X_raw_num_std_scaled)

```

```

Result of scaling the numerical features using standardization:
      longitude  latitude  housing_median_age  total_rooms  total_bedr
ooms \
13096  -1.423037   1.013606                1.861119      0.311912      1.35
9094
14973   0.596394  -0.702103                0.907630     -0.308620     -0.43
6356
3785   -1.203098   1.276119                0.351428     -0.712240     -0.75
9584
14689   1.231216  -0.884924               -0.919891      0.702262      0.73
6231
20507   0.711362  -0.875549                0.589800      0.790125      1.58
5590
...
...
...
14207   0.586397  -0.833359                0.987087     -0.184147      0.13
6961
13105   0.131525   0.319822               -0.443146      0.139847      0.12
5165
19301   1.256209  -1.428701               -1.237721      0.586026      0.55
6922
19121   0.586397  -0.739605                0.669257      0.522417      0.78
8136
19888  -1.418038   0.947978                1.225459     -1.010608     -0.81
1489

```

```

      population  households  median_income
13096    0.137460    1.394812    -0.936491
14973   -0.693771   -0.373485     1.171942
3785    -0.788768   -0.775727   -0.759789
14689    0.383175    0.731375   -0.850281
20507    0.444376    1.755263   -0.180365
...
...
...
14207   -0.445315    0.060101    0.444041
13105   -0.005950    0.083608   -0.685630
19301    1.268299    0.679135    0.101049
19121    0.273563    0.882868    0.145396
19888   -0.679156   -0.752219   -0.310341

```

[16512 rows x 8 columns]

(e) Transformation pipelines [4 marks]

Usually, we have to perform several steps before the data is ready to be fed to an ML algorithm. Scikit-Learn's `ColumnTransformer` and `Pipeline` classes provides a systematic way of "packaging" such a sequence of transformations.

Task: Read the subsection "Transformation Pipelines" in Chapter 2 of the textbook. Then create one pipeline that combines the above steps in the following way applied to the original "raw" features `X_raw`: part (a) (a median imputer for numerical features), part (c) (create the ratios `total_bedrooms/total_rooms`, `total_rooms/households`, `population/households`; and take the log of `total_bedrooms`, `total_rooms`, `population`, `households`, `median_income`), part (d) (a standard scaler for all numerical features, including the transformed features), part (b) (one-hot encoding for categorical variables). Call your transformed features `X`, and print it.

Solution:

```
In [ ]: from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.compose import make_column_selector

def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"] # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),

transformation = ColumnTransformer([("bedrooms_rooms", ratio_pipeline(
    "rooms_per_house", ratio_pipeline(), ["total_rooms", "househo
    "people_per_house", ratio_pipeline(), ["population", "househo
    "log", log_pipeline, ["total_bedrooms", "total_rooms", "popul
    "cat", cat_pipeline, make_column_selector(dtype_include=objec
    remainder=default_num_pipeline) #"drop"

X = transformation.fit_transform(X_raw)
X_df = pd.DataFrame(X, columns=transformation.get_feature_names_out(),
```

```
print("X shape: ", X.shape)
print("X feature names: ", transformation.get_feature_names_out())
print("X: ", X_df)
```

```
X shape: (16512, 16)
X feature names: ['bedrooms_rooms__ratio' 'rooms_per_house__ratio'
'people_per_house__ratio' 'log__total_bedrooms' 'log__total_rooms'
'log__population' 'log__households' 'log__median_income'
'cat__ocean_proximity_<1H OCEAN' 'cat__ocean_proximity_INLAND'
'cat__ocean_proximity_ISLAND' 'cat__ocean_proximity_NEAR BAY'
'cat__ocean_proximity_NEAR OCEAN' 'remainder__longitude'
'remainder__latitude' 'remainder__housing_median_age']
X:      bedrooms_rooms__ratio  rooms_per_house__ratio  people_per_ho
use__ratio \
13096          1.846624          -0.866027          -0.
330204
14973          -0.508121           0.024550          -0.
253616
3785          -0.202155          -0.041193          -0.
051041
14689          -0.149006          -0.034858          -0.
141475
20507           0.963208          -0.666554          -0.
306148
...
...
...
14207           0.804368          -0.452111          -0.
269780
13105          -0.192328           0.036792          -0.
073741
19301          -0.242492          -0.109987           0.
158542
19121           0.259775          -0.360937          -0.
210332
19888           3.612710          -1.323972           0.
049584
```

```
      log__total_bedrooms  log__total_rooms  log__population \
13096          1.324114          0.637892          0.456906
14973          -0.252671          -0.063576          -0.711654
3785          -0.925266          -0.859927          -0.941997
14689           0.952773           0.943475           0.670700
20507           1.437622           1.003590           0.719093
...
...
...
14207           0.465581           0.109720          -0.247054
13105           0.454022           0.477277           0.314542
19301           0.824206           0.859552           1.243381
19121           0.987851           0.811293           0.579462
19888          -1.072179          -2.091543          -0.679447
```

```
      log__households  log__median_income  cat__ocean_proximity_<1H OC
```

EAN \		
13096	1.310369	-1.071522
0.0		
14973	-0.142030	1.194712
1.0		
3785	-0.913030	-0.756981
0.0		
14689	0.925373	-0.912253
0.0		
20507	1.481464	0.034537
0.0		
...	...	...
...		
14207	0.379471	0.648953
1.0		
13105	0.402773	-0.637675
0.0		
19301	0.889897	0.333540
0.0		
19121	1.023329	0.377051
1.0		
19888	-0.853784	-0.119155
0.0		

	cat__ocean_proximity_INLAND	cat__ocean_proximity_ISLAND \
13096	0.0	0.0
14973	0.0	0.0
3785	1.0	0.0
14689	1.0	0.0
20507	0.0	0.0
...	...	...
14207	0.0	0.0
13105	1.0	0.0
19301	0.0	0.0
19121	0.0	0.0
19888	0.0	0.0

	cat__ocean_proximity_NEAR BAY	cat__ocean_proximity_NEAR OCEAN
\		
13096	1.0	0.0
14973	0.0	0.0
3785	0.0	0.0
14689	0.0	0.0
20507	0.0	1.0
...	...	...
14207	0.0	0.0
13105	0.0	0.0
19301	0.0	1.0
19121	0.0	0.0
19888	0.0	1.0

remainder__longitude	remainder__latitude \
----------------------	-----------------------

13096	-1.423037	1.013606
14973	0.596394	-0.702103
3785	-1.203098	1.276119
14689	1.231216	-0.884924
20507	0.711362	-0.875549
...	...	...
14207	0.586397	-0.833359
13105	0.131525	0.319822
19301	1.256209	-1.428701
19121	0.586397	-0.739605
19888	-1.418038	0.947978

	remainder__housing_median_age
13096	1.861119
14973	0.907630
3785	0.351428
14689	-0.919891
20507	0.589800
...	...
14207	0.987087
13105	-0.443146
19301	-1.237721
19121	0.669257
19888	1.225459

[16512 rows x 16 columns]

2. Fine-tuning models [16 marks]

Hint: Reading Chapter 2 of the textbook and the accompanying [Jupyter notebook](#) on the textbook website may help with this question.

(a) Linear regression [2 marks]

Task: Use the `sklearn.linear_model.LinearRegression` class to fit a linear regression to the training set in Question 1. Use the processed features `X` that you obtained in 1(e) as predictors and `y` (the median house values) as the response. Return fitted values of the response for the first 10 observations of the training set.

Solution:

```
In [ ]: from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)
X_predictions10 = lin_reg.predict(X[:10,])
```

```
print("First 10 prediction:")
print(X_predictions[0])
print("Coefficients of Linear Regression model: ")
print(lin_reg.coef_)
print("Intercept of Linear Regression model: ")
print(lin_reg.intercept_)
```

First 10 prediction:

```
[260051.36004114 357897.79846769 125817.08984353 110019.21093156
 298948.54915206 206437.53193813 306787.013961 326902.92850959
 162297.74181859 68786.66157655]
```

Coefficients of Linear Regression model:

```
[ 19022.14390195  7339.05950609  7542.37954629 -32257.13918254
 35372.54029853 -79879.90511328  78574.74380927  65523.70075839
 -31858.81718956 -69970.29802884 173681.40964786 -37867.03776795
 -33985.25666151 -60121.60837206 -63498.61563377 12355.95215798]
```

Intercept of Linear Regression model:

```
251343.44937947788
```

(b) RMSE and MAE [2 marks]

We can measure the performance with root mean square error (RMSE) and mean absolute error (MAE).

Task: Use `mean_squared_error` and `mean_absolute_error` functions from `sklearn.metrics` to calculate the in-sample RMSE and MAE of the linear regression that you fit in part (a).

Solution:

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error
```

```
X_predictions = lin_reg.predict(X)
```

```
X_predict_RMSE = np.sqrt(mean_squared_error(y, X_predictions))
```

```
print("In-sample RMSE: ")
```

```
print(X_predict_RMSE)
```

```
X_predict_MAE = mean_absolute_error(y, X_predictions)
```

```
print("In-sample MAE: ")
```

```
print(X_predict_MAE)
```

In-sample RMSE:

```
70630.22169174395
```

In-sample MAE:

```
53034.28396497742
```

(c) Cross validation [3 marks]

In part (b), we evaluated the in-sample performance of the linear regression model. To obtain out-of-sample performance, we use K-fold cross validation.

Task: Use the `cross_val_score` function from `sklearn.model_selection` to perform 10-fold cross validation on the linear regression from part (a) and return the values of **RMSE**. Return the 10 RMSE scores as well as their mean.

Solution:

```
In [ ]: from sklearn.model_selection import cross_val_score

X_RMSE_cv = np.sqrt(-cross_val_score(lin_reg, X, y, scoring="neg_mean_
X_RMSE_cv10 = X_RMSE_cv[:10]
X_RMSE_cv_mean = np.mean(X_RMSE_cv)
print("10 RMSE scores:")
print(X_RMSE_cv10)
print("Mean of RMSE scores:")
print(X_RMSE_cv_mean)
```

10 RMSE scores:

```
[72183.9055149  69449.07233135 68141.82330925 82348.92814938
 70200.08524327 70974.12115166 70512.16910063 73276.85042458
 69387.52951303 71463.70091766]
```

Mean of RMSE scores:

```
71793.81856557202
```

(d) Alternatives to linear regression [3 marks]

Let us consider fitting two alternative models, namely, **decision trees** and **random forests**. These models will be discussed in future lectures. The following code fits a decision tree and obtains the fitted response for the first 10 observations.

```
In [ ]: from sklearn.tree import DecisionTreeRegressor
dt_reg = DecisionTreeRegressor(random_state=42)
dt_reg.fit(X,y)

dt_reg10 = dt_reg.predict(X[:10]) #,
print(dt_reg10)
```

```
[458300. 483800. 101700. 96100. 361800. 92600. 349300. 440900. 16010
0.
183900.]
```

The following code does the same for the random forest model.

```
In [ ]: from sklearn.ensemble import RandomForestRegressor

rf_reg = RandomForestRegressor(bootstrap=True, n_estimators=100, max_d
rf_reg.fit(X, y)

rf_reg10 = rf_reg.predict(X[:10]) #,
print(rf_reg10)
```

```
[226052.57663663 408572.3162281 100944.59292661 95012.98416211
323566.08786234 135098.37280907 283371.7148996 357336.26089032
167697.90412093 146436.2407077 ]
```

Task: Find the in-sample performance of the decision tree and the random forest using RMSE on the training set. Also, find the out-of-sample performance by 10-fold cross validation as you did in part (c). Which of the 3 models has the best in-sample performance? Which has the best out-of-sample performance?

Solution:

```
In [ ]: # In-sample performance of the decision tree
dtreg_insample = dt_reg.predict(X)
RMSE_dt = np.sqrt(mean_squared_error(y, dtreg_insample))
print("In-sample performance of the Decision Tree (RMSE): ", RMSE_dt)

print('')
# In-sample performance of the random forest
rfreg_insample = rf_reg.predict(X)
RMSE_rf = np.sqrt(mean_squared_error(y, rfreg_insample))
print("In-sample performance of the Random Forest (RMSE): ", RMSE_rf)

print('')
# Out-of-sample performance of the decision tree
dt_outsample_CV10 = -cross_val_score(dt_reg, X, y, scoring="neg_root_m
print("Out-of-sample performance by 10-fold cross validation of the De
print("Average of the Decision Tree (RMSE) by 10-fold cross validation

print('')
# Out-of-sample performance of the random forest
rf_outsample_CV10 = -cross_val_score(rf_reg, X, y, scoring="neg_root_m
print("Out-of-sample performance by 10-fold cross validation of the Ra
print("Average of the Random Forest (RMSE) by 10-fold cross validation
```

In-sample performance of the Decision Tree (RMSE): 0.0

In-sample performance of the Random Forest (RMSE): 62145.4331743811

Out-of-sample performance by 10-fold cross validation of the Decision Tree (RMSE for each fold): [67898.40100744 67248.54149778 67243.39802711 72035.287263

68898.44321855 71008.46519973 71362.02678113 74060.25775818

71016.06635454 69991.86971249]

Average of the Decision Tree (RMSE) by 10-fold cross validation: 70076.27568199515

Out-of-sample performance by 10-fold cross validation of the Random Forest (RMSE for each fold): [64285.5681693 64416.01160789 61285.39123064 62717.91644372

62975.51019993 64267.50742936 64049.97630137 64749.28015876

63792.05556927 64654.70321247]

Average of the Random Forest (RMSE) by 10-fold cross validation: 63719.39203227045

```
In [ ]: ## In-sample performance
print("In-sample performance")
print("Linear Regression (RMSE): ", X_predict_RMSE)
print("Decision Tree (RMSE): ", RMSE_dt)
print("Random Forest (RMSE): ", RMSE_rf)

print('')
## Out-of-sample performance
print("Out-of-sample performance")
print("Linear Regression (RMSE):", X_RMSE_cv_mean)
print("Decision Tree (RMSE for each fold): ", np.mean(dt_outsample_CV1
print("Random Forest (RMSE for each fold): ", np.mean(rf_outsample_CV1
```

In-sample performance

Linear Regression (RMSE): 70630.22169174395

Decision Tree (RMSE): 0.0

Random Forest (RMSE): 62145.4331743811

Out-of-sample performance

Linear Regression (RMSE): 71793.81856557202

Decision Tree (RMSE for each fold): 70076.27568199515

Random Forest (RMSE for each fold): 63719.39203227045

Solution:

Through the 3 models the model that has the best in-sample performance is the Decision Tree model and the model that has the best out-of-sample performance is the Random Forest model as they have the lowest RMSE throughout.

(e) Choosing optimal values of hyperparameters

using cross validation [4 marks]

Most ML algorithms have **hyperparameters**. These hyperparameters control the learning process and should not be confused with the model parameters (which are the unknown values that are being estimated). For example, the random forest algorithm has hyperparameter including `n_estimators` and `max_features`. To fine-tune a model, we should find good values of the hyperparameters based on out-of-sample performance.

Task: Read the subsections "Grid Search" and "Randomized Search" in Chapter 2 of the textbook. Consider the random forest model with the argument `random_state=42`. Use the `sklearn.model_selection.GridSearchCV` class to choose the best hyperparameter values for `max_features` over the values 4, 6, 8, 10 and `n_estimators` over the values 10, 50, 100. Use 3-fold cross-validation with RMSE when searching for the best hyperparameters.

Next, use the `sklearn.model_selection.RandomizedSearchCV` class to perform a randomized search with 10 iterations, where `max_features` is a uniform random integer between 2 and 20 and `n_estimators` is a uniform random integer between 10 and 200. `RandomizedSearchCV` must include the argument `random_state=42`.

In both functions, you may use the argument `n_jobs=-1` to use all available processor cores to speed up the calculation. Based on all of your results, what are the best hyperparameter values?

Solution:

```
In [ ]: from sklearn.model_selection import GridSearchCV
        from sklearn.pipeline import Pipeline

        #full_pipeline = Pipeline([("preprocessing", transformation), ("random
        param_grid = {'max_features': [4, 6, 8, 10], 'n_estimators': [10, 50,
        grid_search = GridSearchCV(RandomForestRegressor(random_state=42), par
        grid_search.fit(X, y)
        best_params = grid_search.best_params_
        print("The best hyperparameter values when using Grid Search are: ", b

        #cv_res = pd.DataFrame(grid_search.cv_results_)
        #cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
```

The best hyperparameter values when using Grid Search are: {'max_featu
res': 6, 'n_estimators': 100}

```
In [ ]: from sklearn.model_selection import RandomizedSearchCV
        from scipy.stats import randint
```

```

param_distribs = {'max_features': randint(low=2, high=20), 'n_estimators':
random_search = RandomizedSearchCV(RandomForestRegressor(random_state=
random_search.fit(X, y)
random_best_params = random_search.best_params_
print("The best hyperparameter values when using Randomized Search are

```

The best hyperparameter values when using Randomized Search are: {'max_features': 7, 'n_estimators': 139}

In []: *# Find the overall best method to choose the best hyperparameter value*

```

best_gridscore = grid_search.best_score_
best_randomscore = random_search.best_score_
print("MSE of 'GridSearchCV': ", -best_gridscore, "; MSE of 'Randomize

if -best_gridscore < -best_randomscore:
    best_model1 = grid_search.best_estimator_
    best_methodparams = best_params
    best_method = "GridSearchCV"
else:
    best_model1 = random_search.best_estimator_
    best_methodparams = random_best_params
    best_method = "RandomizedSearchCV"

print("Best method: ", best_method, "; Best hyperparameter values: ",

```

MSE of 'GridSearchCV': 49466.88231792714 ; MSE of 'RandomizedSearchCV': 49321.2436339602

Best method: RandomizedSearchCV ; Best hyperparameter values: {'max_features': 7, 'n_estimators': 139}

As the "RandomizedSearchCV" method has a lower number of mean squared error (MSE) compared to the "GridSearchCV" method, the

"RandomizedSearchCV" method was more accurate for finding the best hyperparameter values which would be {'max_features': 7, 'n_estimators': 139}.

(f) Evaluating your final model using the test set [2 marks]

The last step of a ML project is to evaluate your (fine-tuned) model using the test set. The goal of this step is to estimate the out-of-sample performance of your trained model on new data. **Be careful not to train your model using the test set.** Do not use `.fit()` or `.fit_transform()` of your estimators or pipelines. Also, **do not fine tune your hyperparameters after finding out the performance of a model on the test set as that would be data snooping.**

Task: Using the test set, find the RMSE of your best model in part (e).

Solution:

```
In [ ]: best_model = random_search.best_estimator_
X_test = strat_test_set.drop("median_house_value", axis=1)
X_test = transformation.transform(X_test)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = best_model.predict(X_test)

final_rmse = np.sqrt(mean_squared_error(y_test, final_predictions))
print("RMSE of best model: ", final_rmse)
```

RMSE of best model: 49016.24364007325

3. Regularizing linear regression [6 marks]

Consider the housing data set from Questions 1 and 2.

(a) Polynomial regression and regularizing [2 marks]

Fit a model that adds degree 3 polynomial terms to the linear regression in 2(a).
Next, regularize the model using ridge regression with parameter $\alpha = 1$.

Solution:

```
In [ ]: from sklearn.preprocessing import PolynomialFeatures

poly = make_pipeline(PolynomialFeatures(degree=3, include_bias=False),
X_poly = poly.fit_transform(X)

poly_reg = LinearRegression()
poly_reg.fit(X_poly, y)
```

```
Out [ ]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
In [ ]: from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=1.0, solver='cholesky')
ridge_reg.fit(X_poly, y)
```

Out []:

▼ Ridge ⓘ ⓘ

Ridge(solver='cholesky')

(b) Learning curves [4 marks]

Obtain learning curves using the function

`sklearn.model_selection.learning_curve` to plot the learning curve with 5-fold CV for the linear regression model that you fitted in 2(a), the polynomial regression model in 3(a), and the ridge regression model in 3(a). Hence, comment on whether the curves indicate the models are underfitting, overfitting or neither.

Solution:

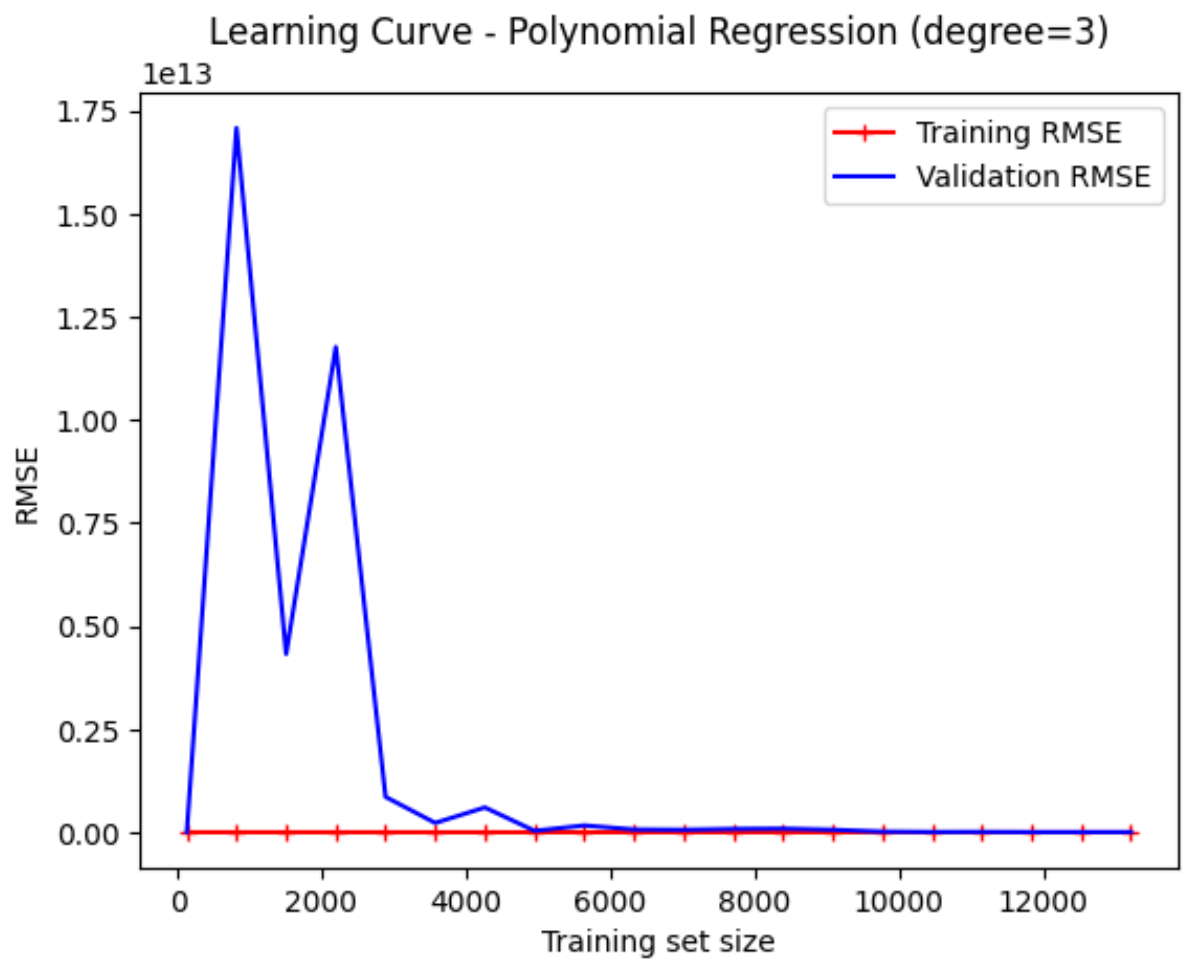
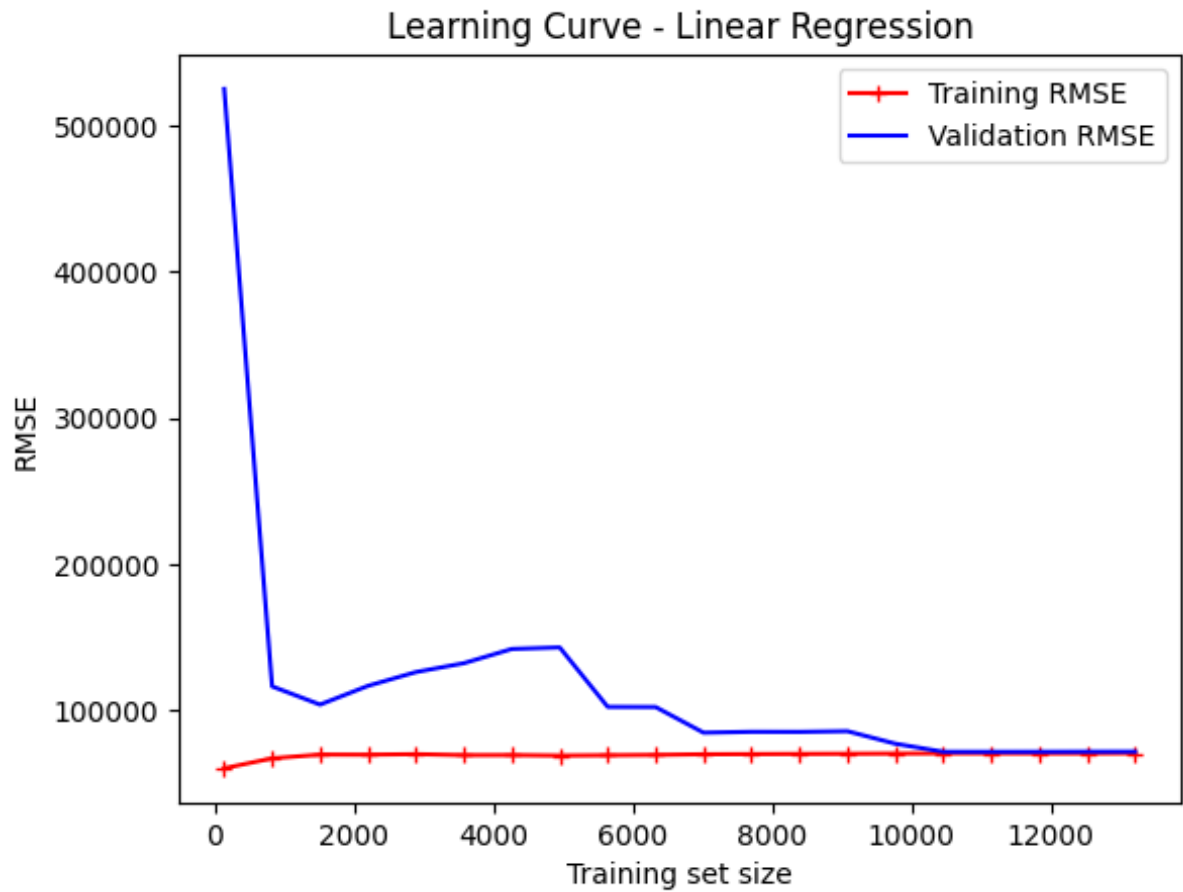
```
In [ ]: import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve

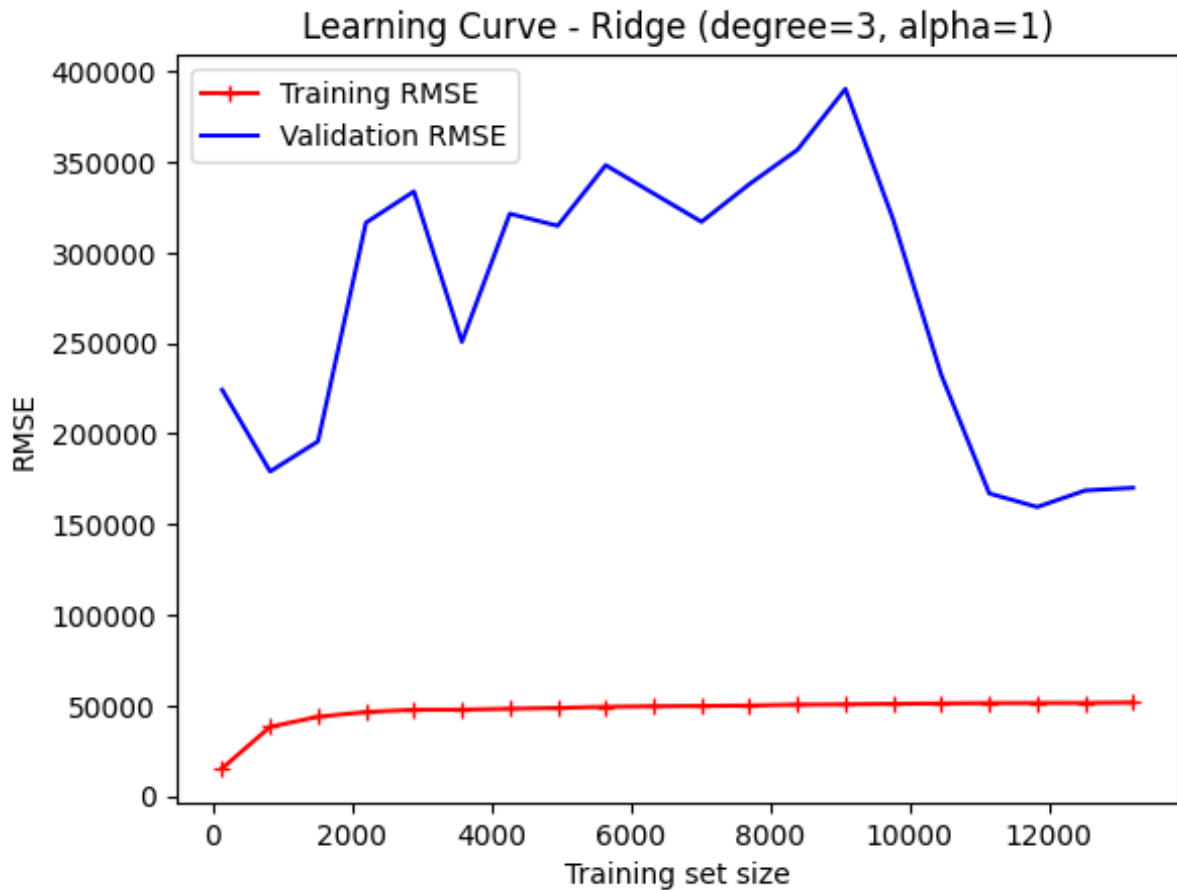
def plot_learning_curve(model, X, y, title, ylim=None):
    train_sizes, train_scores, val_scores = learning_curve(model, X, y)
    train_rmse = -np.mean(train_scores, axis=1)
    val_rmse = -np.mean(val_scores, axis=1)

    #print(train_rmse)
    #print(val_rmse)

    plt.plot(train_sizes, train_rmse, "r--", label="Training RMSE")
    plt.plot(train_sizes, val_rmse, "b--", label="Validation RMSE")
    plt.title(title)
    plt.xlabel("Training set size"); plt.ylabel("RMSE")
    if ylim:
        plt.ylim(ylim)
    plt.legend()
    plt.show()

plot_learning_curve(lin_reg, X, y, "Learning Curve - Linear Regression")
plot_learning_curve(poly_reg, X_poly, y, "Learning Curve - Polynomial")
plot_learning_curve(ridge_reg, X_poly, y, "Learning Curve - Ridge (deg
```





Solution:

The learning curves does indicate the models are underfitting, overfitting or neither. For the learning curve of linear regression, this model is underfitting because the model does not get a low RMSE on both the training and validation set which means it is not capturing the pattern of the data completely. For the learning curve of the polynomial regression, it is neither overfitting or underfitting because both RMSE of the training and validation set is ultimately converging near to 0. For the learning curve of the ridge regression, it is overfitting because the validation RMSE is much higher than the training RMSE which means it is not converging and is overfitting.

4. Default dataset [17 marks]

In this question you will work with the Default dataset, which is a simulated dataset of credit card default for 10000 customers. Download the data as a csv file from [Canvas](#). The data was originally obtained from the R library ISLR2, and you can read the documentation for the dataset [here](#).

You want to predict credit card defaults. The target for this classification problem

is `default` and all other variables are the features.

(a) [2 marks]

Load the dataset. If it is stored in a file named `Default.csv` in your working directory, this can be done with the code below. Then use

`sklearn.model_selection.train_test_split` to split the data into a training and a test set, using 20% of the data for the test set. When doing the splitting, use the random seed 42.

```
In [ ]: import os
data = pd.read_csv("Default.csv")
```

Solution:

```
In [ ]: from sklearn.model_selection import train_test_split

X = data.drop("default", axis=1)
y = data["default"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
print("X train: ")
print(X_train)
print("y train: ")
print(y_train)
```

```
X train:
      student      balance      income
9254      No  1018.568130  34103.879520
1561      No   62.170050  28660.747508
1670      No  1046.743543  40822.447413
6087      No   763.735280  44125.718725
6669     Yes   697.248633  25730.917583
...      ...      ...      ...
5734      No   865.697004  33541.046380
5191     Yes   493.629546  20500.212632
5390      No   722.029162  26984.256287
860      Yes   163.939823  21083.004510
7270      No   637.338107  49657.394282
```

```
[8000 rows x 3 columns]
```

```
y train:
```

```
9254      No
1561      No
1670      No
6087      No
6669      No
...
5734      No
5191      No
5390      No
860      No
7270      No
```

```
Name: default, Length: 8000, dtype: object
```

(b) [3 marks]

Design a pipeline for preprocessing that applies one hot encoding to the categorical features and a standard scaler to the numerical features.

Solution:

```
In [ ]: from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer

num_pipeline = make_pipeline(StandardScaler())

cat_pipeline = make_pipeline(OneHotEncoder(handle_unknown="ignore"))

transformation = ColumnTransformer([
    ("num", num_pipeline, X.select_dtypes(include=["int64", "float64"]
    ("cat", cat_pipeline, X.select_dtypes(include=["object"])).columns.
],
remainder="drop")

X_train_trans = transformation.fit_transform(X_train)
```

```

X_test_trans = transformation.transform(X_test)

X_train_final = pd.DataFrame(X_train_trans, columns=['balance', 'income', 'student_no', 'student_yes'])
X_test_final = pd.DataFrame(X_test_trans, columns=['balance', 'income', 'student_no', 'student_yes'])
print("X train: ")
print(X_train_final)
print("X test: ")
print(X_test_final)

```

X train:

	balance	income	student_no	student_yes
0	0.381082	0.037308	1.0	0.0
1	-1.591774	-0.370099	1.0	0.0
2	0.439202	0.540178	1.0	0.0
3	-0.144587	0.787420	1.0	0.0
4	-0.281735	-0.589390	0.0	1.0
...	...	...	...	...
7995	0.065740	-0.004819	1.0	0.0
7996	-0.701760	-0.980897	0.0	1.0
7997	-0.230618	-0.495580	1.0	0.0
7998	-1.381844	-0.937276	0.0	1.0
7999	-0.405319	1.201454	1.0	0.0

[8000 rows x 4 columns]

X test:

	balance	income	student_no	student_yes
0	1.241465	-0.157056	1.0	0.0
1	-0.127973	0.638724	1.0	0.0
2	-1.720019	-0.882921	1.0	0.0
3	-1.485744	-0.059998	1.0	0.0
4	1.081535	1.219882	1.0	0.0
...	...	...	...	...
1995	1.303734	-1.192078	0.0	1.0
1996	0.275237	-0.105071	1.0	0.0
1997	-1.663700	-0.640293	1.0	0.0
1998	1.062606	-0.953199	0.0	1.0
1999	-1.103820	1.196101	1.0	0.0

[2000 rows x 4 columns]

(c) [4 marks]

Train a logistic regression classifier and a `SGDClassifier`, and then using 10-fold cross-validation plot the ROC curve of the classifiers and compute their ROC AUC. Use the default arguments, except you must include the argument `random_state=42`. Furthermore, for the logistic regression, compute the ROC AUC for `C` taking the values 0.001, 0.01, 0.1, 1, 10, 100, to show that the default value of 1 is approximately optimal.

Solution:

```

In [ ]: from sklearn.linear_model import LogisticRegression, SGDClassifier
        from sklearn.metrics import roc_curve, roc_auc_score
        from sklearn.model_selection import cross_val_predict

log_reg = LogisticRegression(random_state=42)
sgd_clf = SGDClassifier(random_state=42)

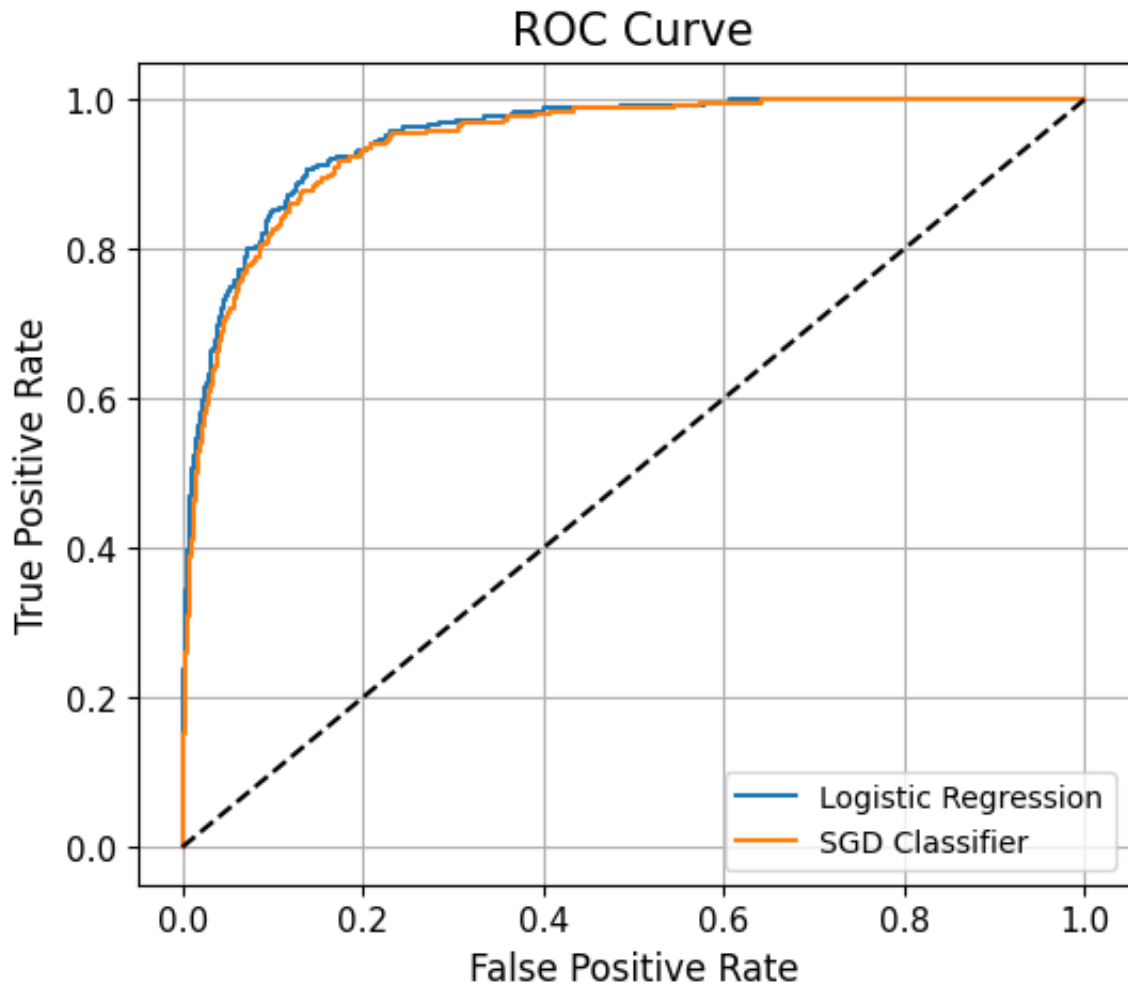
y_scores_log = cross_val_predict(log_reg, X_train_final, y_train, cv=1)
y_scores_sgd = cross_val_predict(sgd_clf, X_train_final, y_train, cv=1)
y_train = y_train.map({'Yes': 1, 'No': 0})
y_test = y_test.map({'Yes': 1, 'No': 0})

fpr_log, tpr_log, threshold_log = roc_curve(y_train, y_scores_log)
fpr_sgd, tpr_sgd, threshold_sgd = roc_curve(y_train, y_scores_sgd)

plt.figure(figsize=(6,5))
plt.plot(fpr_log, tpr_log, label="Logistic Regression")
plt.plot(fpr_sgd, tpr_sgd, label="SGD Classifier")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate", fontsize='12')
plt.ylabel("True Positive Rate", fontsize='12')
plt.title("ROC Curve", fontsize='15')
plt.tick_params(axis='both', which='major', labelsize=11)
plt.legend()
plt.grid()
plt.show()

auc_log = roc_auc_score(y_train, y_scores_log)
auc_sgd = roc_auc_score(y_train, y_scores_sgd)
print(f"ROC AUC (Logistic Regression): {auc_log}")
print(f"ROC AUC (SGD Classifier): {auc_sgd}")

```



ROC AUC (Logistic Regression): 0.9501484597787597

ROC AUC (SGD Classifier): 0.9433360557488014

```
In [ ]: from sklearn.model_selection import GridSearchCV

# Define the parameter grid with different C values.
param_grid = {"C": [0.001, 0.01, 0.1, 1, 10, 100]}

# Setup a GridSearchCV object with scoring based on ROC AUC.
grid_search = GridSearchCV(
    LogisticRegression(random_state=42, max_iter=10000),
    param_grid,
    cv=10,
    scoring="roc_auc",
    return_train_score=True
)

# Fit the grid search on the training data.
grid_search.fit(X_train_final, y_train)

# Get the best parameters and score.
best_C = grid_search.best_params_["C"]
best_score = grid_search.best_score_
```

```

print("Best C based on ROC AUC:", best_C)
print("Highest score during cross validation on ROC AUC:", best_score)

# If desired, you can also inspect the full results.
results = grid_search.cv_results_
mean_score_l = []
for mean_score, std_score, param in zip(results["mean_test_score"],
                                         results["std_test_score"],
                                         results["params"]):
    print(f"{param}: {mean_score} ± {std_score:.4f}")
    mean_score_l.append(mean_score)

# Plot
plt.figure(figsize=(8, 6))
plt.semilogx([0.001, 0.01, 0.1, 1, 10, 100], mean_score_l, marker='o')
plt.xlabel("C")
plt.ylabel("Cross-Validated ROC AUC")
plt.title("C vs Cross validation score based on Logistic Regression RO
plt.grid(True)
plt.axvline(1, color='gray', linestyle='--', label='Default C=1')
plt.legend()
plt.show()

```

Best C based on ROC AUC: 1

Highest score during cross validation on ROC AUC: 0.9512133568768846

{'C': 0.001}: 0.9471556770618115 ± 0.0195

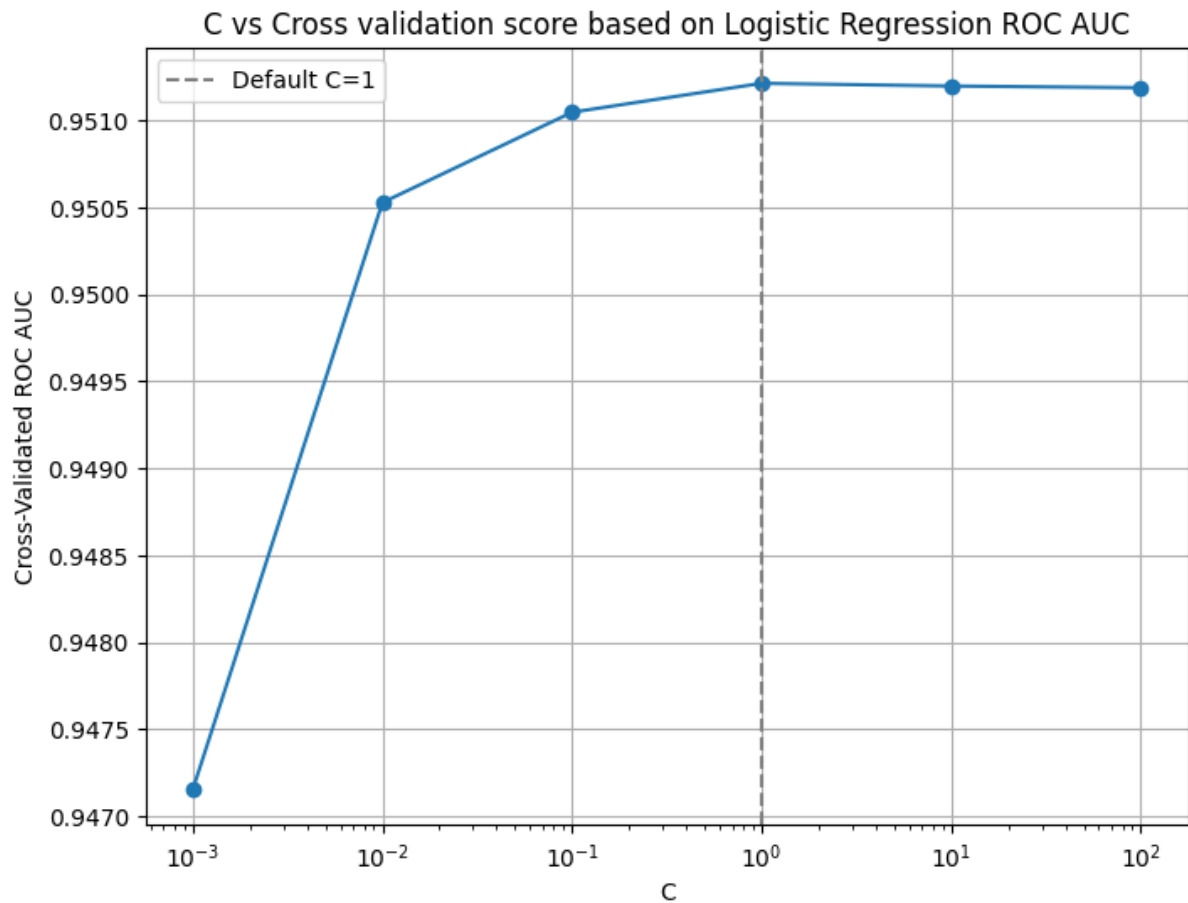
{'C': 0.01}: 0.9505280070805509 ± 0.0189

{'C': 0.1}: 0.9510470721886529 ± 0.0188

{'C': 1}: 0.9512133568768846 ± 0.0189

{'C': 10}: 0.9511977378888237 ± 0.0190

{'C': 100}: 0.9511879773605457 ± 0.0190



Solution:

Above graph and cross validation score shows the default value, $C = 1$ is approximately optimal.

(d) [5 marks]

For the classifiers in part (c), plot on the same axes the precision as a function of the threshold and the recall as a function of the threshold. Now suppose you want to achieve a recall of 75%. Consider 4 models:

1. The logistic regression classifier in (c),
2. `SGDClassifier` in (c),
3. A logistic regression with the argument `class_weight="balanced"`,
4. `SGDClassifier` with the argument `class_weight="balanced"`.

For each of the 4 models, determine the threshold to achieve a 75% recall based on 10-fold cross-validation. Then determine the corresponding precisions. Which model has the best precision?

Solution:

```

In [ ]: from sklearn.metrics import precision_recall_curve

# Model 1
precisions_log, recalls_log, thresholds_log = precision_recall_curve(y

plt.figure(figsize=(8,5))
plt.plot(thresholds_log, precisions_log[:-1], "b--", label="Precision")
plt.plot(thresholds_log, recalls_log[:-1], "r-", label="Recall")
plt.axhline(y=0.75, color='orange', linestyle='--', label="75%")
plt.xlabel("Threshold", fontsize=12)
plt.ylabel("Precision / Recall", fontsize=12)
plt.legend(fontsize=12, loc="center left")
plt.title("Precision / Recall vs. Threshold for Logistic Regression",

# Model 2
precisions_sgd, recalls_sgd, thresholds_sgd = precision_recall_curve(y

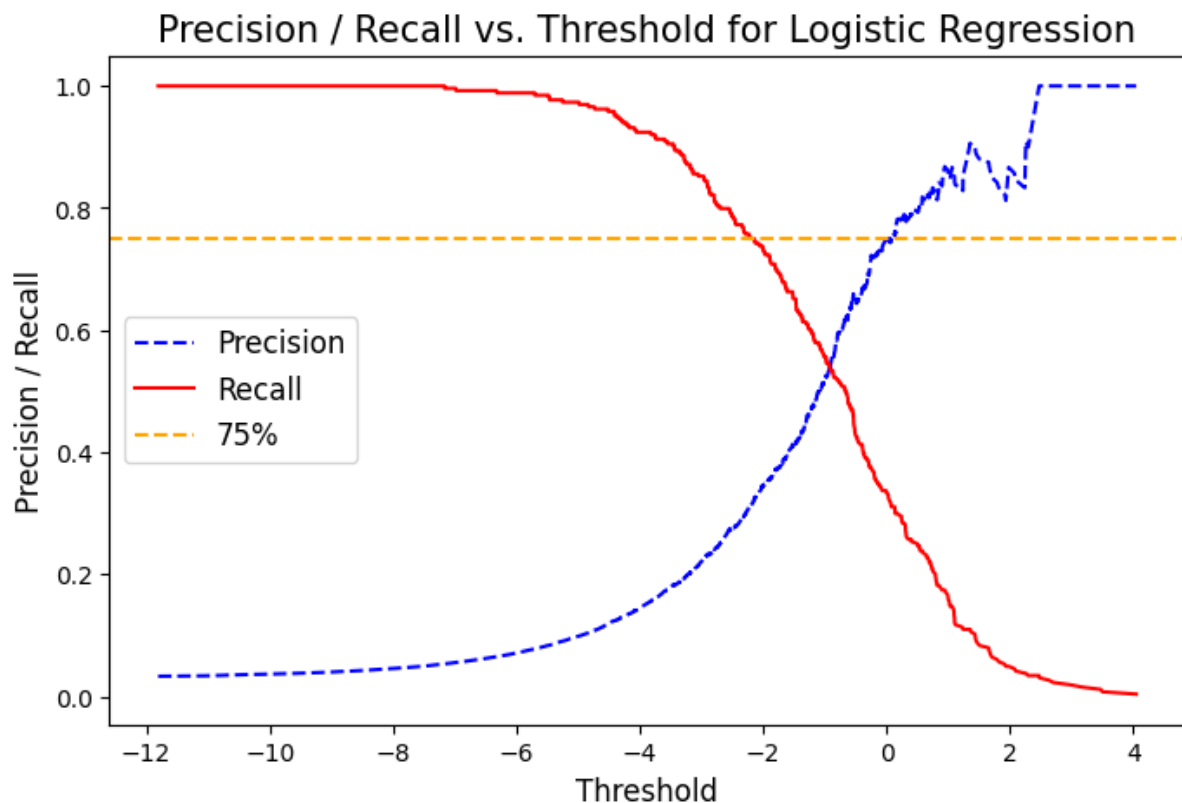
plt.figure(figsize=(8,5))
plt.plot(thresholds_sgd, precisions_sgd[:-1], "b--", label="Precision")
plt.plot(thresholds_sgd, recalls_sgd[:-1], "r-", label="Recall")
plt.axhline(y=0.75, color='orange', linestyle='--', label="75%")
plt.xlabel("Threshold", fontsize=12)
plt.ylabel("Precision / Recall", fontsize=12)
plt.legend(fontsize=12)
plt.title("Precision / Recall vs. Threshold for SGD Classifier", fonts

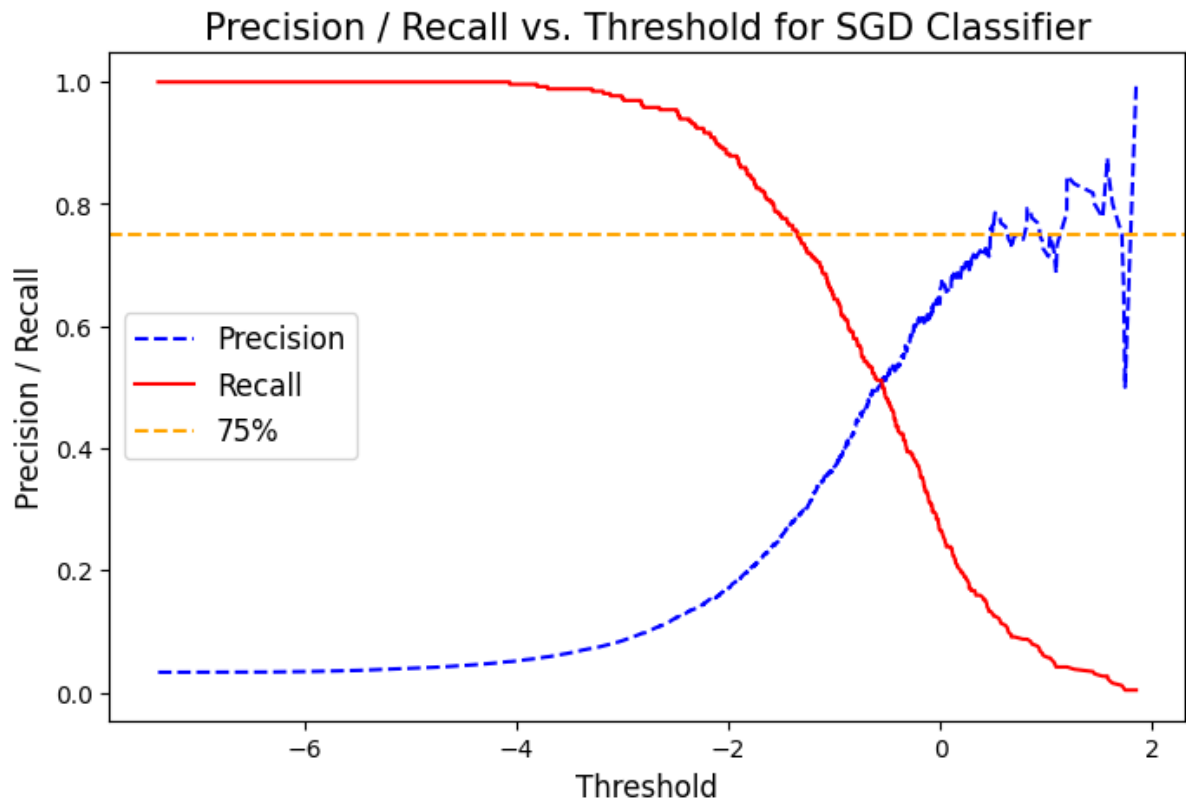
```

```

Out[ ]: Text(0.5, 1.0, 'Precision / Recall vs. Threshold for SGD Classifier')

```



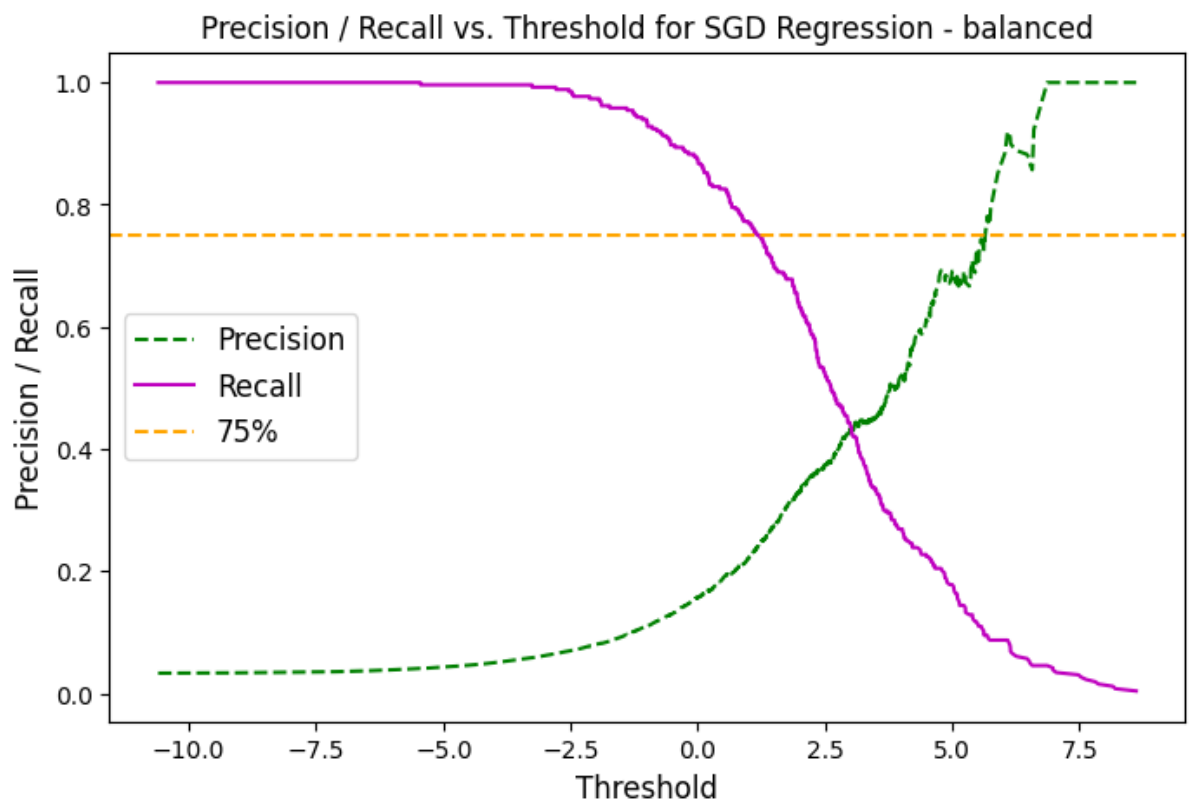
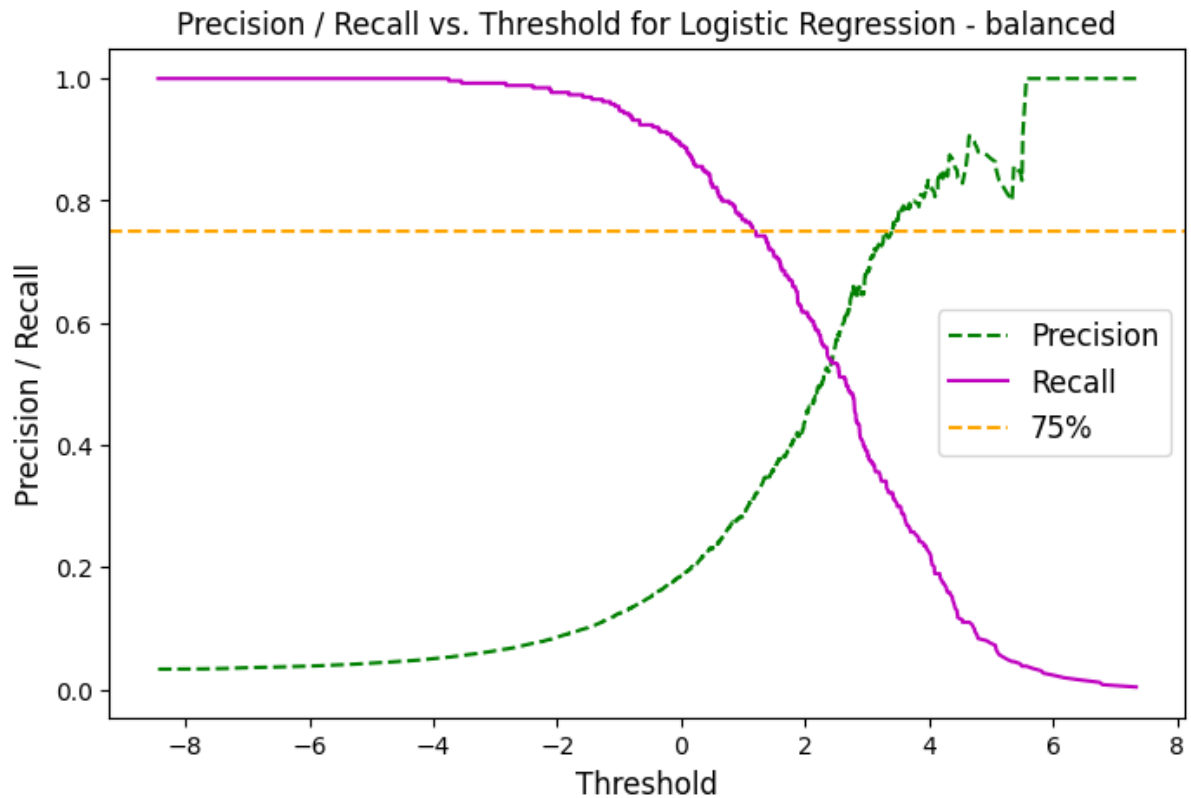


```
In [ ]: log_reg_bal = LogisticRegression(random_state=42, class_weight="balanced")
sgd_clf_bal = SGDClassifier(random_state=42, class_weight="balanced")
y_scores_log_bal = cross_val_predict(log_reg_bal, X_train_final, y_train_final)
y_scores_sgd_bal = cross_val_predict(sgd_clf_bal, X_train_final, y_train_final)
precisions_log_bal, recalls_log_bal, thresholds_log_bal = precision_recall_curve(y_train_final, y_scores_log_bal)
precisions_sgd_bal, recalls_sgd_bal, thresholds_sgd_bal = precision_recall_curve(y_train_final, y_scores_sgd_bal)

# Model 3
plt.figure(figsize=(8,5))
plt.plot(thresholds_log_bal, precisions_log_bal[:-1], "g--", label="Precision")
plt.plot(thresholds_log_bal, recalls_log_bal[:-1], "m-", label="Recall")
plt.axhline(y=0.75, color='orange', linestyle='--', label="75%")
plt.xlabel("Threshold", fontsize=12)
plt.ylabel("Precision / Recall", fontsize=12)
plt.legend(fontsize=12)
plt.title("Precision / Recall vs. Threshold for Logistic Regression - balanced")

# Model 4
plt.figure(figsize=(8,5))
plt.plot(thresholds_sgd_bal, precisions_sgd_bal[:-1], "g--", label="Precision")
plt.plot(thresholds_sgd_bal, recalls_sgd_bal[:-1], "m-", label="Recall")
plt.axhline(y=0.75, color='orange', linestyle='--', label="75%")
plt.xlabel("Threshold", fontsize=12)
plt.ylabel("Precision / Recall", fontsize=12)
plt.legend(fontsize=12, loc="center left")
plt.title("Precision / Recall vs. Threshold for SGD Regression - balanced")
```

```
Out[ ]: Text(0.5, 1.0, 'Precision / Recall vs. Threshold for SGD Regression - balanced')
```



```
In [ ]: def precision_at_threshold(precisions, recalls, thresholds):
        index = np.argmax(recalls <= 0.75)
        threshold = thresholds[index]
        precision = precisions[index]
        return threshold, precision
```

```

thres_log, precision_at_thres_log = precision_at_threshold(precisions_
print("Model 1. The logistic regression in (c)")
print("The threshold to achieve a 75% recall based on 10-fold cross-v
print("The corresponding precisions: ", precision_at_thres_log)

thres_sgd, precision_at_thres_sgd = precision_at_threshold(precisions_
print("Model 2. The SGD classifier in (c)")
print("The threshold to achieve a 75% recall based on 10-fold cross-v
print("The corresponding precisions: ", precision_at_thres_sgd)

thres_log_bal, precision_at_thres_log_bal = precision_at_threshold(pre
print("Model 3. The logistic regression with the argument class_weigh
print("The threshold to achieve a 75% recall based on 10-fold cross-v
print("The corresponding precisions: ", precision_at_thres_log_bal)

thres_sgd_bal, precision_at_thres_sgd_bal = precision_at_threshold(pre
print("Model 4. The SGD classifier with the argument class_weight='ba
print("The threshold to achieve a 75% recall based on 10-fold cross-v
print("The corresponding precisions: ", precision_at_thres_sgd_bal)

```

Model 1. The logistic regression in (c)
The threshold to achieve a 75% recall based on 10-fold cross-validation
n: -2.2140476615696842
The corresponding precisions: 0.3108320251177394
Model 2. The SGD classifier in (c)
The threshold to achieve a 75% recall based on 10-fold cross-validation
n: -1.3484894484634338
The corresponding precisions: 0.2877906976744186
Model 3. The logistic regression with the argument class_weight='balanced'
The threshold to achieve a 75% recall based on 10-fold cross-validation
n: 1.153871874409318
The corresponding precisions: 0.309375
Model 4. The SGD classifier with the argument class_weight='balanced'
The threshold to achieve a 75% recall based on 10-fold cross-validation
n: 1.1590314281116108
The corresponding precisions: 0.23826714801444043

Solution:

The model that has the best precision is first model which is the Logistic Regression model that is not balanced and has the precision approximately 0.310832.

(e) [3 marks]

Fit the best classifier in (d) to the whole training data, and use it to predict on the test set. What is its accuracy, precision, recall, ROC curve and ROC AUC over the test set?

Solution:

```

In [ ]: best_model = LogisticRegression(random_state=42)
best_model.fit(X_train_final, y_train)

y_test_prediction = best_model.predict(X_test_final)

from sklearn.metrics import accuracy_score, precision_score, recall_score

accuracy = accuracy_score(y_test, y_test_prediction)
precision = precision_score(y_test, y_test_prediction)
recall = recall_score(y_test, y_test_prediction)

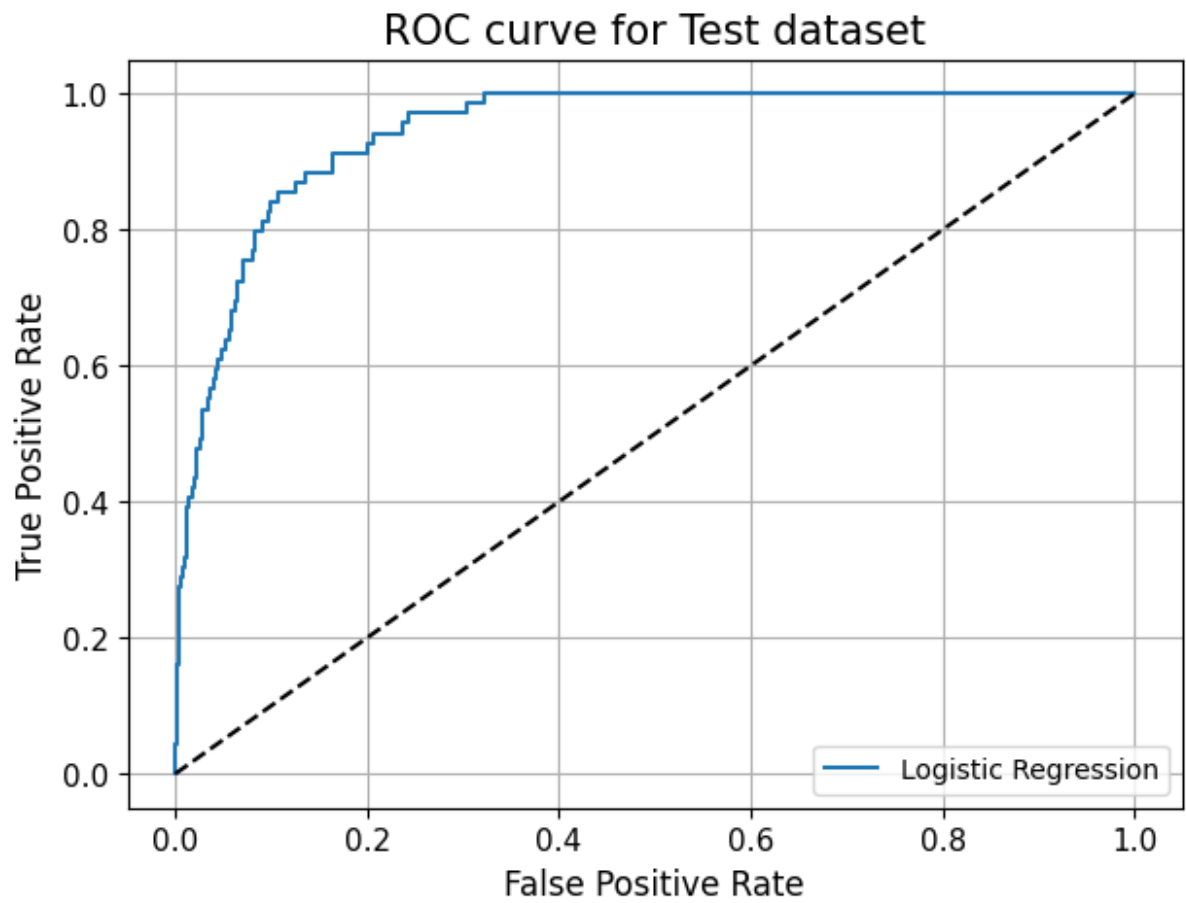
print("Accuracy over the test set: ", accuracy)
print("Precision over the test set: ", precision)
print("Recall over the test set: ", recall)

ytest_scores_log = best_model.predict_proba(X_test_final)[:, 1]
fpr_log_test, tpr_log_test, threshold_log_test = roc_curve(y_test, ytest_scores_log)
print("ROC curve over the test set: ")
plt.figure(figsize=(7,5))
plt.plot(fpr_log_test, tpr_log_test, label="Logistic Regression")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate", fontsize='12')
plt.ylabel("True Positive Rate", fontsize='12')
plt.title("ROC curve for Test dataset", fontsize='15')
plt.tick_params(axis='both', which='major', labelsize=11)
plt.legend()
plt.grid()
plt.show()

auc_log_test = roc_auc_score(y_test, ytest_scores_log)
print(f"ROC AUC over the test set: {auc_log_test}")

```

Accuracy over the test set: 0.97
 Precision over the test set: 0.6551724137931034
 Recall over the test set: 0.2753623188405797
 ROC curve over the test set:



ROC AUC over the test set: 0.9434925209585783